

Tema: Tamaño de lote

A continuación se provee una descripción del proceso de entrenamiento de una red neuronal donde se pretende dar un entendimiento de la contribución de la cantidad de muestras disponibles para este proceso. Para esta descripción se toma como ejemplo el conjunto de muestras ilustrado en la [Figura 1](#). También, se representa con una línea continua lo que podrían ser los límites de las distribuciones del conjunto de muestras en cada clase (En este caso dos clases), es decir, se considera que en el supuesto caso de existir nuevas muestras (adicionales a las ya mostradas en la [Figura 1](#)), no deberían aparecer fuera de estos límites, esto denotaría un conjunto de entrenamiento adecuado para el modelo de red. Como en todos los ejercicios de clasificación, se desea encontrar la función matemática que represente una frontera de decisión que separe las clases. Idealmente, esta frontera debería existir en una posición lo mas alejada de cada uno de los límites de las distribuciones por clase, para el caso de una clasificación, se podría entender que la frontera debería posicionarse junto en medio de ambas distribuciones.

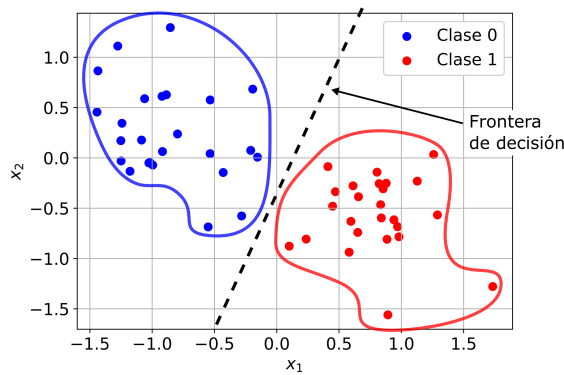


Figura 1: Conjunto de muestras etiquetado en dos clases tomado como ejemplo para explicar el tamaño de lote.

Entonces, se definirá como *lote* al conjunto de muestras empleadas para calcular un costo y donde a partir de esta función de costo se obtienen sus derivadas con respecto a todos los parámetros de la red para poder realizar una actualización de los mismos. En los ejemplos vistos hasta el momento se han utilizado todas las muestras para realizar este procedimiento, es decir, el tamaño de lote (denotado como ν en lo subsecuente) ha sido igual al número de muestras. En el caso de clasificaciones binarias una de las funciones de costo empleadas ha sido la entropía cruzada binaria expresada hasta el momento como:

$$C = -\frac{1}{m} \sum_{i=1}^m \sum_{d=1}^{S_L} (T_d^{(i)} \log P_d^{(i)} + (1 - T_d^{(i)}) \log(1 - P_d^{(i)})) \quad (1)$$

donde $S_L = 2$ para este ejemplo (modelo de red con dos neuronas en su capa de salida) y entendiendo que esta expresión aplica para modelos de red que cuentan en su capa de salida con funciones de activación sigmoide. De esta expresión se puede notar que el costo depende de las m muestras. Sin embargo, si se reflexiona lo ya mencionado, en principio, no debería existir la necesidad de utilizar todas las muestras para poder representar la frontera de decisión. La idea detrás de esto es que se podría seleccionar un subconjunto de muestras que puedan representar los mismos límites de distribución que el conjunto de muestras completo de una clase ([Figura 2a](#)) o en su defecto, puede existir un subconjunto de muestras por clase que, aunque representen una distribución por clase distinta a aquella dada por el total de muestras, sea aun posible conseguir una frontera que exitosamente separe las clases considerando la totalidad de muestras ([Figura 2b](#)).

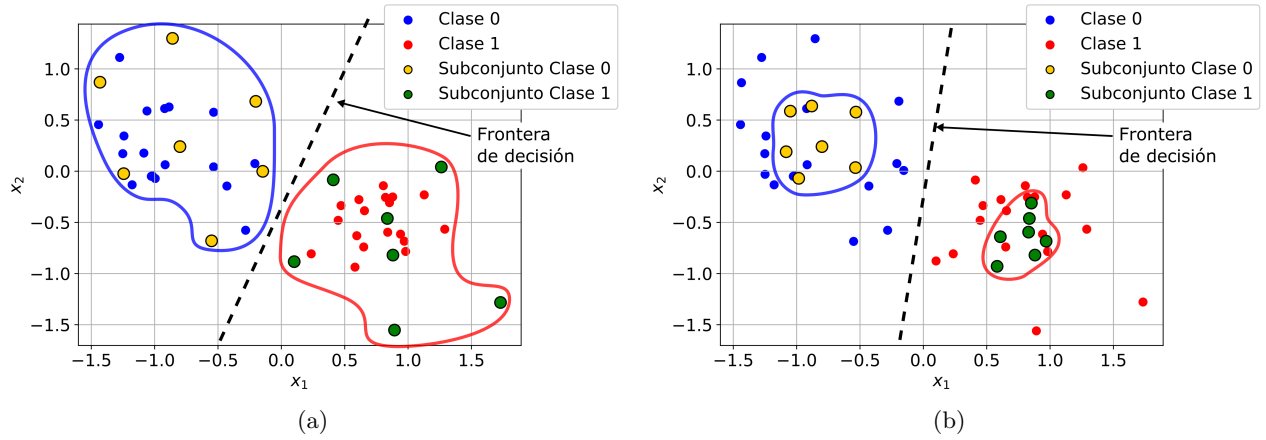


Figura 2: Representación de una frontera de decisión dada por un subconjunto de muestras por clase. a) Subconjunto de muestras con límites de distribución equivalente a aquellos dados por la totalidad de muestras b) Subconjunto de muestras con límites de distribución distintos a los originales. En este escenario la frontera separa correctamente ambas clases considerando todas las muestras.

Es importante señalar que lo descrito para la Figura 2 ejemplifica escenarios con subconjuntos de muestras seleccionadas aleatoriamente con distribución uniforme pero con una cantidad de muestras equitativa para ambas clases. No obstante existe la posibilidad de que en esta selección aleatoria de muestras no exista igualdad en la cantidad de muestras seleccionadas por clase. Esto provocaría que la frontera se posicionara en un lugar donde la clasificación podría ser correcta para los subconjuntos de muestras por clase pero errónea para el total de las muestras. Tal es caso de lo que se ilustra en la Figura 3a, donde el desbalance en la cantidad de muestras para los subconjunto causa que la clase 1 no se clasifique correctamente. Un caso extremo se muestra en la Figura 3b, donde solo existe una muestra para el subconjunto de la clase 0, lo cual favorece la correcta clasificación de la clase 1 pero es un escenario desfavorable para la correcta clasificación de la clase 0.

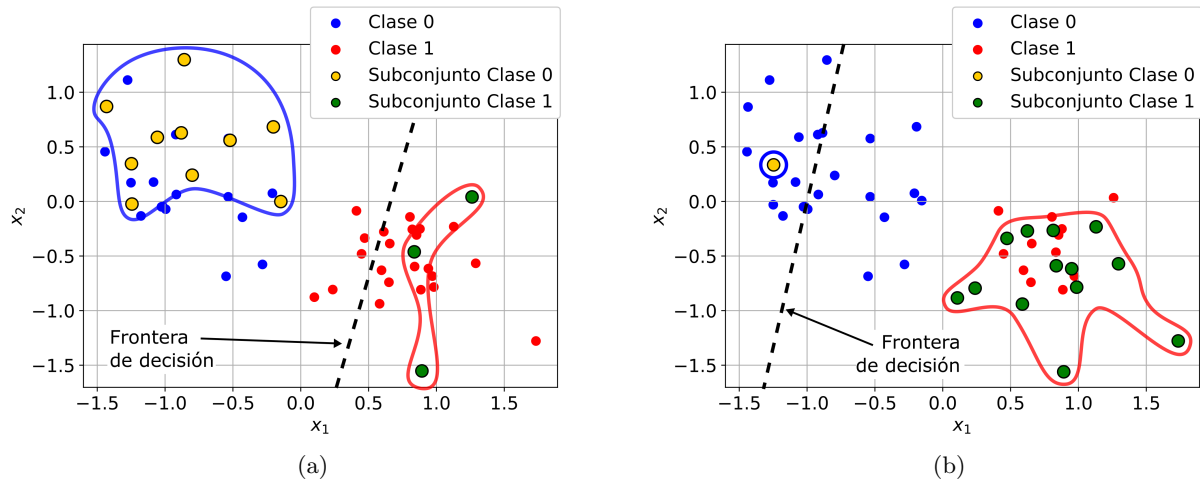


Figura 3: Representación de una frontera de decisión dada por un subconjunto de muestras por clase en escenarios donde la cantidad de muestras no es igual en cada subconjunto.

De estos ejemplos se pudiera interpretar [Figura 3](#) la correcta clasificación viene dada no por la cantidad de muestras empleadas en el entrenamiento, sino por que tan fiel es la distribución de los subconjuntos con respecto a la distribución de las muestras para cada clase.

Para ilustrar el efecto de emplear diferente número de muestras en el entrenamiento se realiza el ejercicio de clasificación siguiendo la siguiente metodología:

1. Se toma como conjunto de entrenamiento aquel mostrado en la [Figura 1](#).
2. Se considera la siguiente hipótesis:

$$p^{(i)} = \frac{1}{1 + e^{-(x_1^{(i)} \omega_1 + x_2^{(i)} \omega_2)}} \quad (2)$$

esto, dado a que contiene solo dos parámetros entrenables que implica el poder visualizar el hiperplano de la función de costo.

3. se toma como función de costo:

$$C = -\frac{1}{\nu} \sum_{i=1}^{\nu} (T^{(i)} \log P^{(i)} + (1 - T^{(i)}) \log(1 - P^{(i)})) \quad (3)$$

donde se asigna para el valor de ν tres posibles valores: $\nu = m$, $\nu = 25$ y $\nu = 1$.

4. Se establece como regla de actualización de los pesos la dada por GD:

$$\omega_{j,t+1} = \omega_{j,t} - \alpha \frac{\partial C}{\partial \omega_{j,t}} \quad (4)$$

5. Se establecen como parámetros de entrenamiento los siguientes valores:

- (a) $\omega_1 = -8$ y $\omega_2 = 8$
- (b) $\alpha = 0.4$
- (c) Épocas = 200

El termino *época* se refiere al hecho de haber empleado la totalidad de las muestras durante el entrenamiento, mientras que las *iteraciones* es la cantidad de veces que se actualizan los pesos. Por ejemplo: Si se tienen 50 muestras, y $\nu = 50$ entonces una época equivale a una iteración y por tanto se realiza una actualización de los pesos ω_1 y ω_2 en cada época. En cambio, si $\nu = 10$, significa que se utilizarán 10 muestras para el calculo del costo y derivadas de tal modo que ocurrirán cinco actualizaciones de los pesos ω_1 y ω_2 para haber completado el uso de todas las muestras, en otras palabras, una epoca tendrá cinco iteraciones.

6. Primeramente, se muestran los resultados de emplear un tamaño de lote igual al total de muestras $\nu = m$. Estos resultados comprenden:

- La grafica del costo a través de las épocas/iteraciones
- La ubicación del costo dados los valores por iteración de ω_1 y ω_2 .
- El mapa de probabilidades
- La frontera de decisión

En lo subsecuente se muestran los mismo resultados para $\nu = 25$ y finalmente los correspondientes a $\nu = 25$

Cuando $\nu = m$ se entiende que el entrenamiento de la hipótesis se realiza en *batch* y al emplear el algoritmo GD se denota este método como *Batch Gradient Descent* siendo su procedimiento el que se describe a continuación en el pseudocódigo 1:

Algorithm 1 Pseudocódigo del algoritmo *Batch Gradient Descent*

Require: Conjunto de entrenamiento $\mathbf{A} = \{(\mathbf{x}^{(1)}, \mathbf{T}^{(1)}), \dots, (\mathbf{x}^{(m)}, \mathbf{T}^{(m)})\}$

for $t = 1$ to épocas **do**

- Realizar la propagación hacia adelante para las m muestras del conjunto $\mathbf{A}$ a fin de obtener m predicciones
- Calcular el costo para las m predicciones ($m = \nu$) empleando la [Ecuación 3](#).
- Calcular las derivas de la función de costo con respecto a todos los parámetros ω :

$$\frac{\partial C}{\partial \omega_j} = \frac{1}{m} \sum_{i=1}^m (P^{(i)} - T^{(i)})x_j \quad (5)$$

- Actualizar pesos con la regla expresada en la [Ecuación 4](#)

end for

Cabe señalar que en lo descrito dentro del pseudocódigo 1 se plantea repetir los pasos tantas veces como el valor de épocas lo indique. Esta condición del lazo “for” puede ser sustituida por alguna estrategia que repita los pasos hasta alcanzar la convergencia del valor del costo. Sin embargo, en la mayoría de los casos es suficiente con asignar de manera arbitraria el valor del costo y en caso de no existir convergencia una vez finalizado el entrenamiento se pueden agregar una mayor cantidad de épocas o cambiar los parámetros del entrenamiento.

En la [Figura 4](#) se ilustran los resultados del entrenamiento usando *Batch Gradient Descent* ($\nu = m$). La [Figura 4a](#) muestra el costo obtenido en cada época. La característica principal de este método es que el cambio del costo es “suave” conforme incrementa la cantidad de épocas, es decir, el comportamiento aparenta visualmente ser una señal continua. Esta característica permite deducir sin mayor problema si la convergencia existe, lo cual es el caso de lo mostrado en esta [Figura 4a](#). El siguiente resultado es el trayecto que sigue costo (Cuya ubicación esta dado por los valores de ω_1 y ω_2), dentro del hiperplano de la función de costo ([Figura 4b](#)). El costo inicial ($\omega_1 = -8$ y $\omega_2 = 8$) es indicado por una marca roja. Su posición es actualizada conforme se ejecuta el algoritmo de entrenamiento de tal modo que se este costo sigue la trayectoria indicada por la linea azul para finalmente posicionarse donde indica la marca verde (costo final) en la época 200. Así, con los valores de $\omega_1 = -8$ y $\omega_2 = 8$ producidos en esta ultima época es posible generar el mapa de probabilidades mostrado en la [Figura 4c](#) y a partir de este mapa se puede visualizar la frontera de decisión ([Figura 4d](#)). A partir del resultado visual se evidencia una correcta clasificación (precisión de 100%). Estas visualizaciones se toman como referencia para hacer comparación con los resultados obtenidos para $\nu = 25$ y $\nu = 1$.

En el pseudocódigo 2 se describen los pasos para realizar el entrenamiento de un modelo empleando el método *Mini Batch Gradient Descent*. Este se caracteriza por emplear solo un subconjunto de muestras ($1 < \nu < m$) para el calculo de costo, obtención de derivadas y actualización de los pesos. La idea es que al inicio de cada época, el conjunto $\mathbf{A}$ debe revolverse conservado cada etiqueta con su respectivamente, es decir, no se deben revolver independientemente las muestras y las etiquetas. Al realizar este reacomodo de muestras y etiquetas aleatoria para generar el conjunto $\mathbf{B}$ en cada iteración puede provocar el efecto ilustrado

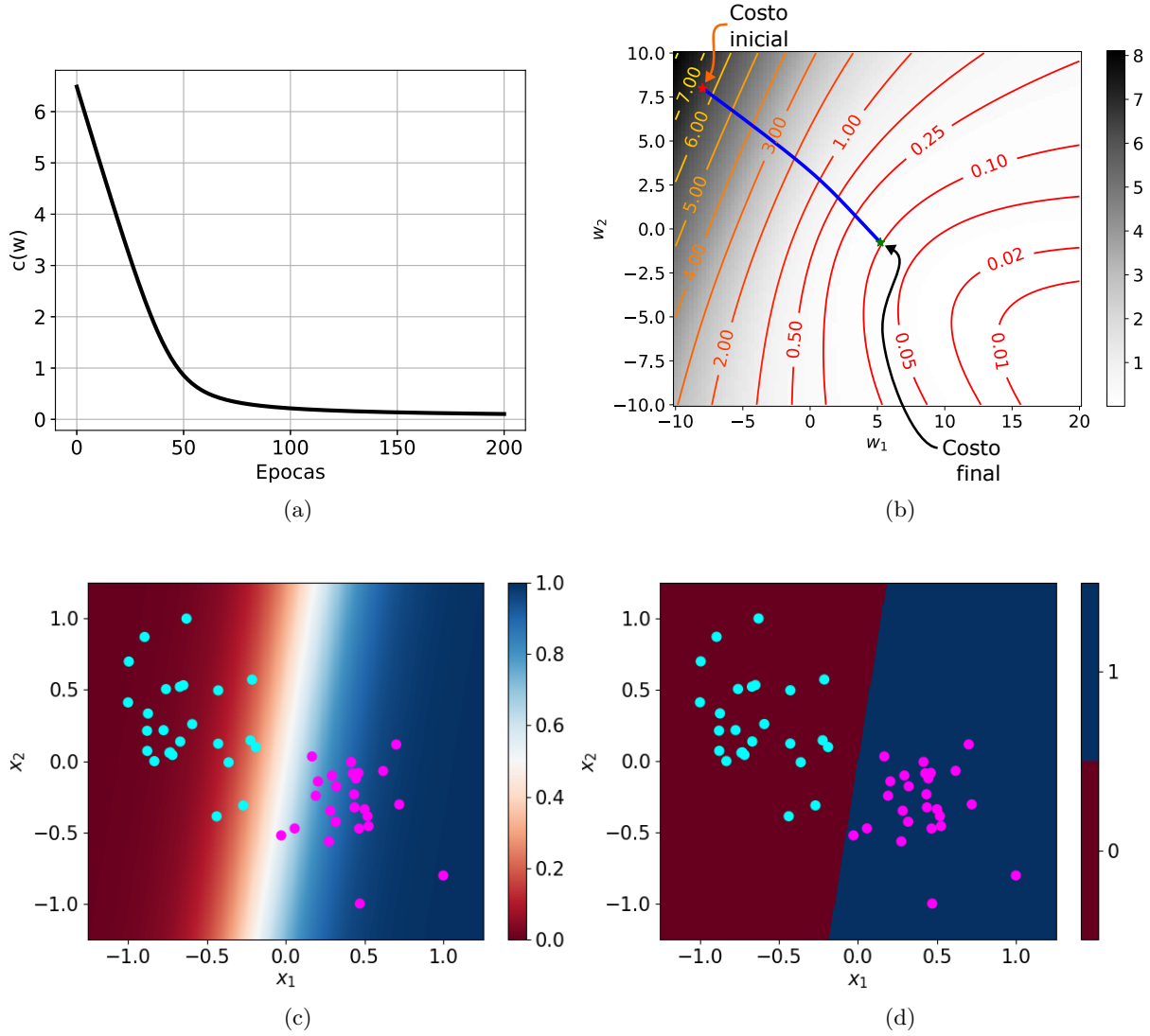


Figura 4: Resultados del algoritmo de entrenamiento basado en *Batch Gradient Descent* a) Costo obtenido por época b) Ubicación del costo (dado por w_1 y w_2) dentro del hiperplano de la función de costo c) Mapa de probabilidades d) Frontera de decisión.

en la Figura 3. Entonces, para este pseudocódigo 2 se podría modificar de tal manera que para la generación de $\mathbf{B}$ se incluyan un cantidad igual de muestras para cada clase. No obstante, esta modificación no suele ser necesaria. Una evidencia de esto son los resultados presentados en la Figura 5 producto de un algoritmo de entrenamiento *Mini Batch Gradient Descent* con $\nu = 25$.

La Figura 5a muestra el costo obtenido en cada iteración, aunque el eje de abscisas está etiquetado como la cantidad de épocas, en realidad existe τ cálculos de costos por época. Se puede observar que el cambio del costo conforme al incremento de las iteraciones ya no es tan continuo sino más bien tiende a contener cierto nivel de aleatoriedad, este comportamiento depende de cuáles muestras son seleccionadas en cada iteración, sin embargo la tendencia es un descenso con comportamiento similar al obtenido por el método basado en *Batch*. Otro cambio significativo es que el descenso en el valor del costo más acelerado; cuando

Algorithm 2 Pseudocódigo del algoritmo *Mini Batch Gradient Descent*

Require:

→ Conjunto de entrenamiento $\mathbf{A} = \{(\mathbf{x}^{(1)}, \mathbf{T}^{(1)}), \dots, (\mathbf{x}^{(m)}, \mathbf{T}^{(m)})\}$

→ ν

→ Determinar número de lotes: $\tau = \lfloor m/\nu \rfloor$

for $t = 1$ to épocas **do**

 Revolver $\mathbf{A}$

for $k = 1$ to τ **do** (Esto corresponde al número de iteraciones)

- Tomar ν muestras del conjunto $\mathbf{A}$ para generar un conjunto $\mathbf{B}$ tal que:

$$\mathbf{B} = \{(\mathbf{x}^{((\nu \times (k-1)) + 1)}, \mathbf{T}^{((\nu \times (k-1)) + 1)}), \dots, (\mathbf{x}^{(\nu k)}, \mathbf{T}^{(\nu k)})\} \quad (6)$$

- Realizar la propagación hacia adelante para las ν muestras del conjunto $\mathbf{B}$ a fin de obtener ν predicciones
- Calcular el costo para las ν predicciones empleando la [Ecuación 3](#).
- Calcular las derivas de la función de costo con respecto a todos los parámetros ω :

$$\frac{\partial C}{\partial \omega_j} = \frac{1}{\nu} \sum_{i=1}^{\nu} (P^{(i)} - T^{(i)}) x_j \quad (7)$$

- Actualizar pesos con la regla expresada en la [Ecuación 4](#)

end for

end for

$\nu = m$ (*Batch*) el valor del costo desciende por debajo de 1 justo antes de la época 50, mientras que en *Mini Batch* ($\nu = 25$) esto mismo ocurre alrededor de la época 25.

La ubicación del costo se ilustra en la [Figura 5b](#). Aunque visualmente no es tan evidentemente, la trayectoria que sigue el costo de igual manera contiene cierto comportamiento aleatorio. Por otra parte, el valor del costo final es menor con respecto al alcanzado por el método en *Batch* ya que se llega a tener un costo menor a 0.1. Finalmente la [Figura 5c](#) y [Figura 5d](#) muestran el mapa de probabilidades y la frontera de decisión, respectivamente. En estas ultimas, nuevamente se evidencia una clasificación con 100% de precisión. La diferencia mas notable con respecto a los resultados con *Batch* es que la frontera decisión no es exactamente la misma. Así, en este punto las ventajas mas claras de *Mini Batch* sobre *Batch* son:

1. Una convergencia del valor del costo en un menor numero de épocas.
2. El valor del costo en la ultima época es menor.
3. El calculo del costo y derivadas se realiza con una menor cantidad de datos, lo que se traduce en un demanda computacional menor (menor cantidad de operaciones en cada iteración).

Estas ventajas mencionadas pueden hasta cierto punto ser mas aprovechadas si se disminuye el valor de ν , siendo el mínimo valor $\nu = 1$. Así, se puede llevar a cabo un entrenamiento siguiendo los pasos descritos en el pseudocódigo [3](#). Este describe un entrenamiento llamado *Stochastic Batch Gradient Descent* que se caracteriza por emplear una muestra al azar del conjunto de entrenamiento para realizar el calculo del costo y derivadas, teniendo en cuenta que la muestra seleccionada al azar no se puede volver a seleccionar dentro de una misma época.

Los resultados de aplicar este ultimo método estocástico se presentan en la [Figura 6](#). Como es de esperarse, al disminuir el tamaño de lote al mínimo se logra una disminución del valor del costo mas acelerado ([Figura 6a](#)), la desventaja en este aspecto es que es costo en cada nueva iteración puede variar significativamente, y en modelos con un gran número de parámetros podría llegar a ser complicado determinar si existe la

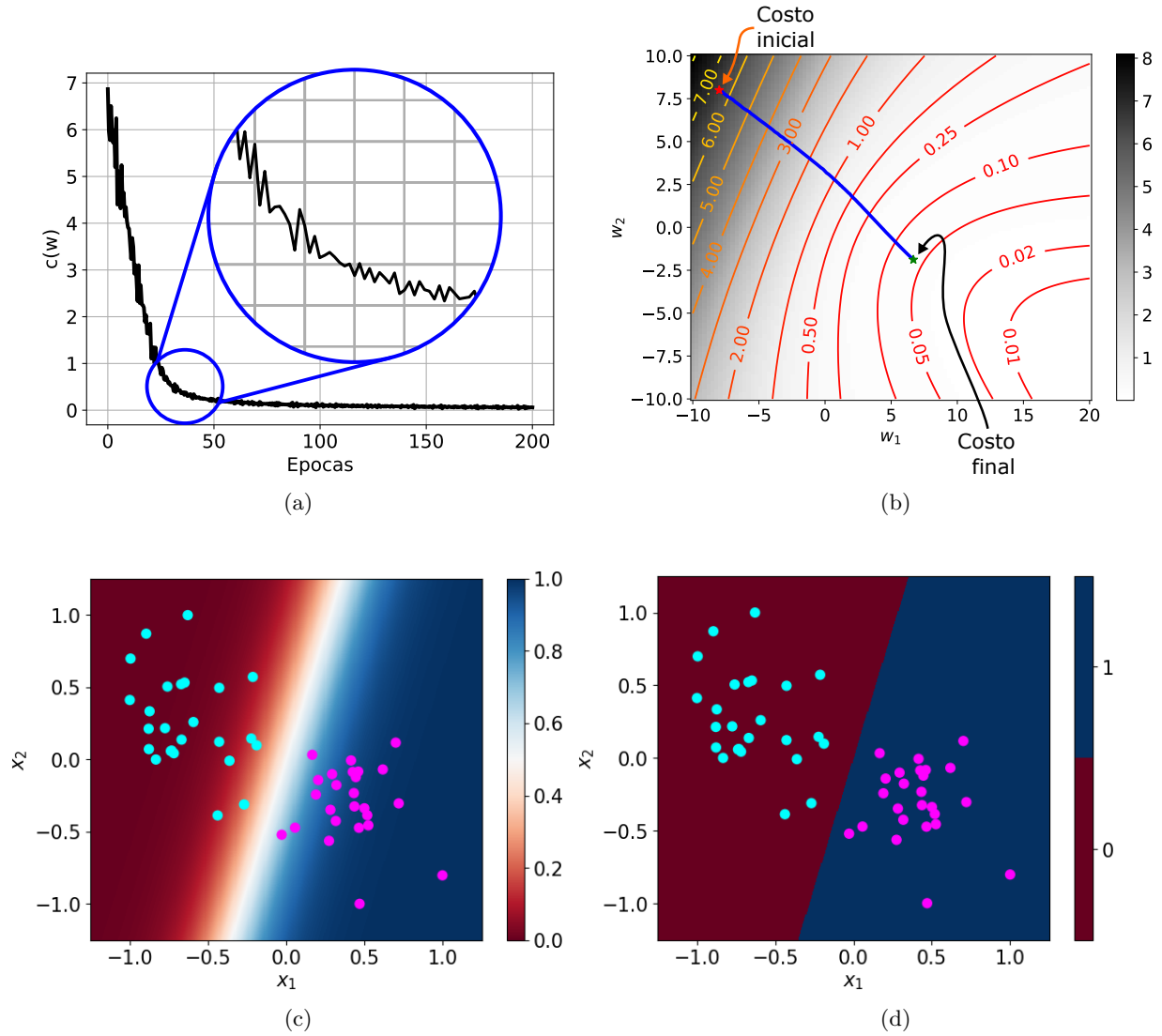


Figura 5: Resultados del algoritmo de entrenamiento basado en *Mini Batch Gradient Descent* a) Costo obtenido por época b) Ubicación del costo (dado por ω_1 y ω_2) dentro del hiperplano de la función de costo c) Mapa de probabilidades d) Frontera de decisión.

convergencia o no. Al menos para este ejemplo, se puede evidenciar que el costo si continua disminuyendo en cada nueva época pues el valor del costo logra llegar a ser menor que 0.01 (Figura 6b). Aunque la ubicación de costo realiza un recorrido un tanto “caótico” es posible alcanzar valores de costo menores que los que se obtienen con *Batch* y *Mini Batch* en la misma cantidad de épocas. Este costo menor implica una zona de incertidumbre menos (“mas delgada”) en el mapa de probabilidades (Figura 6c), lo cual es un aspecto benéfico en cualquier clasificador. Por ultimo se tiene la frontera de decisión (Figura 6d) que indica una precisión de 100%

Algorithm 3 Pseudocódigo del algoritmo *Stochastic Batch Gradient Descent*

Require:

→ Conjunto de entrenamiento $\mathbf{A} = \{(\mathbf{x}^{(1)}, \mathbf{T}^{(1)}), \dots, (\mathbf{x}^{(m)}, \mathbf{T}^{(m)})\}$

for $t = 1$ to épocas **do**

 Revolver $\mathbf{A}$

for $k = 1$ to m **do**

- Realizar la propagación hacia adelante para una sola muestra del conjunto $\mathbf{A}$ a fin de obtener una sola predicción
- Calcular el costo a partir de la predicción obtenida empleando la Ecuación 3 ($\nu = 1$).
- Calcular las derivas de la función de costo con respecto a todos los parámetros ω :

$$\frac{\partial C}{\partial \omega_j} = (P^{(i)} - T^{(i)})x_j \quad (8)$$

- Actualizar pesos con la regla expresada en la Ecuación 4
- end for**

end for

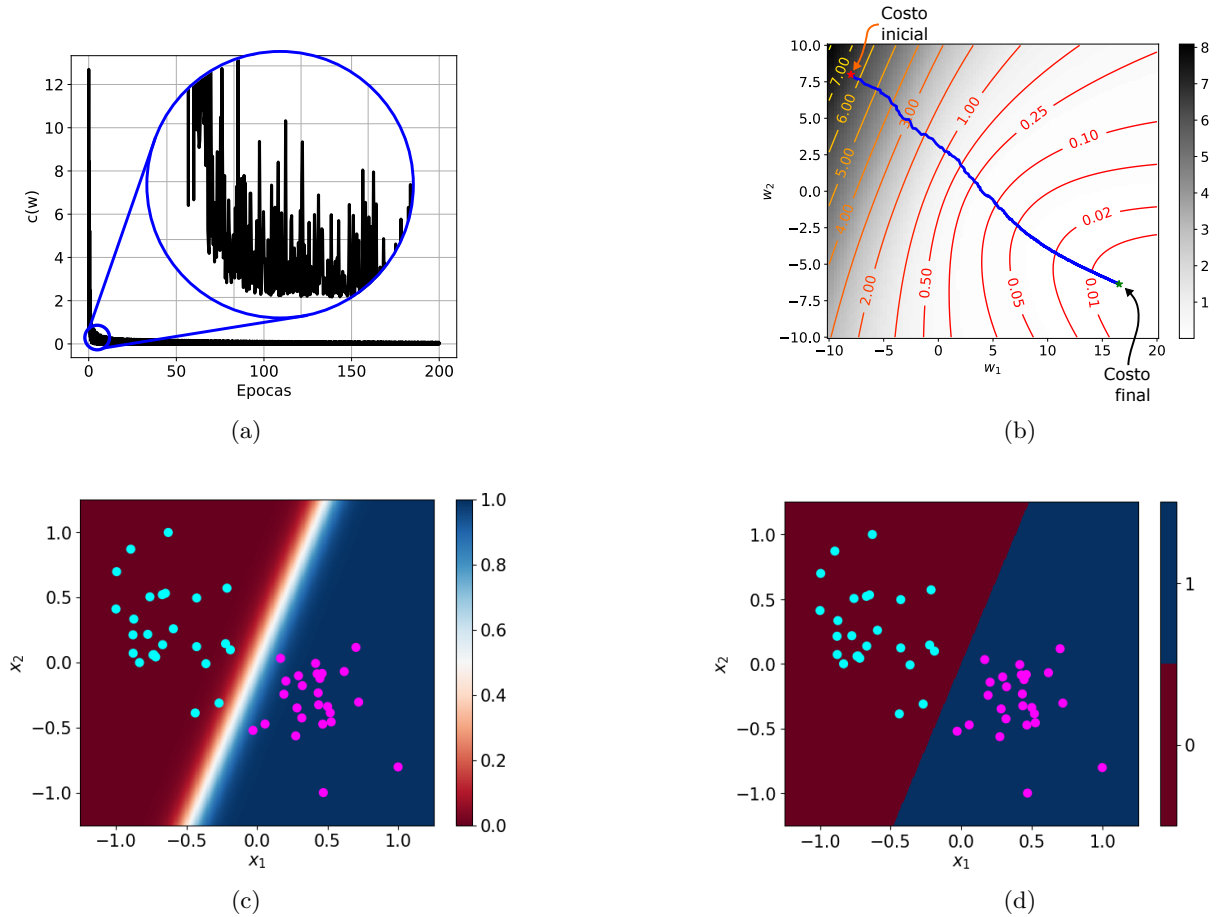


Figura 6: Resultados del algoritmo de entrenamiento basado en *Stochastic Gradient Descent* a) Costo obtenido por época b) Ubicación del costo (dado por ω_1 y ω_2) dentro del hiperplano de la función de costo c) Mapa de probabilidades d) Frontera de decisión.